JUnit Programming Exercises

Name: Rakesh N
Program: Cognizant Digital Nurture 5.0
Topic: JUnit Testing

Exercise 1: Setting Up JUnit

Problem Analysis

The objective is to configure a Java project with the JUnit framework and execute a simple unit test to verify that the testing environment is correctly set up.

Root Cause

Without adding the JUnit library, Java cannot recognize annotations such as `@Test` or assertion methods like `assertEquals()`. As a result, unit tests cannot be executed.

Solution

Maven Dependency

```
<dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>4.13.2</version>
    <scope>test</scope>
</dependency>
```

Test Code

```
import org.junit.Test;
import static org.junit.Assert.*;

public class MyFirstTest {

    @Test
    public void testSample() {
        assertEquals(4, 2 + 2);
```

```
    }
}
```

Output

```
JUnit version 4.13.2
.
Time: 0.012

OK (1 test)
```

Prevention Tips

- Add the correct JUnit dependency before writing tests.
- Use the correct import statements.
- Verify the project builds successfully before running tests.

Exercise 2: Assertions in JUnit

Problem Analysis

Applications require validation to ensure expected and actual values match. JUnit assertions help verify program correctness.

Root Cause

Without assertions, incorrect program behavior may go undetected.

Solution

```
import org.junit.Test;
import static org.junit.Assert.*;

public class AssertionsTest {

    @Test
    public void testAssertions() {

        assertEquals(5, 2 + 3);

        assertTrue(5 > 3);
```

```
        assertFalse(5 < 3);

        assertNull(null);

        assertNotNull(new Object());
    }
}
```

Output

```
JUnit version 4.13.2
.
Time: 0.008

OK (1 test)
```

Prevention Tips

- Select the appropriate assertion method.
- Test both valid and invalid scenarios.
- Keep one logical verification per assertion.

Exercise 3: Arrange–Act–Assert (AAA) Pattern

Problem Analysis

Unit tests become difficult to understand if setup, execution, and verification are mixed together.

Root Cause

Poorly organized test methods reduce readability and maintainability.

Solution

Calculator.java

```
public class Calculator {

    public int add(int a, int b) {
        return a + b;
```

```
    }

    public int subtract(int a, int b) {
        return a - b;
    }
}
```

CalculatorTest.java

```
import org.junit.After;
import org.junit.Before;
import org.junit.Test;

import static org.junit.Assert.*;

public class CalculatorTest {

    private Calculator calculator;

    @Before
    public void setUp() {
        calculator = new Calculator();
        System.out.println("Setting up Calculator instance");
    }

    @Test
    public void testAddition() {

        int result = calculator.add(10, 5);

        assertEquals(15, result);
    }

    @Test
    public void testSubtraction() {

        int result = calculator.subtract(10, 5);

        assertEquals(5, result);
    }

    @After
    public void tearDown() {
        calculator = null;
        System.out.println("Tearing down Calculator instance");
```

```
    }
}
```

Output

```
Setting up Calculator instance
Tearing down Calculator instance
Setting up Calculator instance
Tearing down Calculator instance

JUnit version 4.13.2
..

Time: 0.010

OK (2 tests)
```

Prevention Tips

- Follow the Arrange–Act–Assert structure.
- Use `@Before` for common setup tasks.
- Use `@After` for cleanup.
- Keep test methods independent.
- Use meaningful test method names.

Conclusion

These exercises demonstrate the fundamentals of JUnit testing, including project setup, assertions, and organizing test cases using the Arrange–Act–Assert pattern. Applying these practices improves code quality, simplifies debugging, and increases confidence in software reliability.